

Hospital Management System

Final Technology Stack, Workflow, Architecture, Reporting, AI, and Build Roadmap

Item	Decision
Primary frontend	React + TypeScript + Vite
Primary backend	ASP.NET Core Web API on current .NET LTS
Primary database now	Microsoft SQL Server
Future enterprise database option	Oracle-ready design, not Oracle-first unless client requires it
Medical imaging storage	DICOM/PACS + object storage; database stores metadata and references
Reporting	SSRS/Power BI Report Server + QuestPDF + FastReport/Stimulsoft optional + AI narrative layer
AI phase	Separate Python FastAPI AI service after core HMS is stable
Architecture	Clean Architecture + Modular Monolith first, microservices later only where justified
Prepared date	May 16, 2026

This blueprint is written for planning and implementation. It keeps the hospital workflow first, the technology second, and the AI phase last. The goal is a production-grade hospital management system that can start with one hospital and grow into a multi-branch enterprise system.

Table of Contents

- 1. 1. Executive recommendation
- 2. 2. Final technology stack
- 3. 3. Frontend comparison and UI direction
- 4. 4. Backend comparison and recommendation
- 5. 5. Database strategy: SQL Server now, Oracle-ready later
- 6. 6. Medical imaging and file storage strategy
- 7. 7. Reporting system strategy
- 8. 8. AI technology stack and AI phase
- 9. 9. Full hospital workflow
- 10. 10. Full software architecture
- 11. 11. Security, compliance, and audit design
- 12. 12. DevOps, hosting, Docker, and Kubernetes plan
- 13. 13. Step-by-step build phases and estimated timeline
- 14. 14. Rules for Codex-assisted development
- 15. 15. Next deliverable after confirmation
- 16. 16. References

1. Executive Recommendation

Use ASP.NET Core as the main enterprise backend. It is a strong fit for this project because most of the hospital system is business workflow: patient registration, OPD, admission, bed management, lab, pharmacy, billing, reports, security, audit logs, and integrations. AI is important, but it should not drive the whole architecture from day one.

Use React for the frontend. Build role-based dashboards for reception, doctors, nurses, floor reception, admission, billing, lab, radiology, pharmacy, housekeeping, admin, and patients. Use a premium online template if it saves time, but control the design system yourself so the product does not look like a generic admin dashboard.

Use SQL Server for the first production version. It fits ASP.NET Core very well, makes reporting easier, and speeds up development. Keep the data access layer Oracle-ready so that a large enterprise client can later request Oracle without forcing a rewrite of the full product.

Do not store MRI, CT, X-ray, ultrasound, or scan image files directly inside the main relational database. Use DICOM/PACS and object storage. Store metadata, patient links, report text, DICOM identifiers, file references, and access permissions in the hospital database.

Use a hybrid data access approach: Entity Framework Core for normal workflow and transactions, Dapper for fast read/report queries, and stored procedures only for heavy reports, stock valuation, financial closing, and large summaries.

Build AI at the end, after the hospital management workflow is stable. AI should generate drafts, explanations, summaries, risk alerts, and report narratives. AI should not silently finalize clinical decisions or change medical records without human review.

2. Final Technology Stack

Layer	Recommended Stack	Why
Frontend app	React + TypeScript + Vite	Fast SPA development, strong ecosystem, good for dashboards and role-based portals.
UI components	Material UI/MUI X first; optional Tailwind + shadcn/ui for custom design	MUI is practical for enterprise tables, forms, modals, filters, and dashboards.
State and API data	TanStack Query + Axios/Fetch + React Context/Zustand where needed	Reliable API caching and server-state management.
Forms and validation	React Hook Form + Zod/Yup	Complex hospital forms need consistent validation.
Charts	Recharts or ECharts	Dashboards for patient count, bed occupancy, revenue, stock, and lab workload.
Main backend	ASP.NET Core Web API on current .NET LTS	Enterprise-grade, secure, high performance, excellent for SQL Server and Microsoft environments.
Architecture	Clean Architecture + Modular Monolith	Keeps modules separated without microservice overhead.

ORM/data access	EF Core + Dapper + Stored Procedures for heavy reporting	Best balance of speed, maintainability, and performance.
Database now	Microsoft SQL Server	Best fit for ASP.NET, reporting, enterprise workflows, and MVP speed.
Database future option	Oracle support through separated infrastructure/data provider layer	Use when a client requires Oracle or enterprise Oracle infrastructure.
Cache	Redis	Queue counters, dashboard caching, sessions if needed, rate limiting, and AI/result cache.
Real-time	SignalR	Live queues, bed board updates, nurse notifications, lab/pharmacy events.
Background jobs	Hangfire first; Quartz.NET if complex scheduling is needed	SMS, email, report generation, reminders, exports, AI jobs.
Reporting engine	SSRS/Power BI Report Server + QuestPDF + optional FastReport/Stimulsoft	Covers enterprise reports, printable documents, and designer-based reports.
Medical images	PACS/DICOM server + DICOMweb + object storage	Correct architecture for MRI/CT/X-ray image storage and retrieval.
Object storage	MinIO on-prem, S3/Azure Blob/R2 cloud	Stores files, reports, consent forms, documents, DICOM objects if integrated.
AI service	Python FastAPI + OpenAI/Azure OpenAI/local models + vector search	AI belongs in a separate service for flexibility and safety.
AI/ML imaging later	MONAI/PyTorch for validated imaging AI experiments	Only after clinical validation and governance.
Containers	Docker for all app services	Repeatable development, staging, production images.
Orchestration	Kubernetes after MVP/staging; not required on day one	Useful for scaling services, workers, AI service, and high availability.
Logging/monitoring	Serilog + OpenTelemetry + Prometheus/Grafana or Application Insights + Sentry	Production observability and issue tracking.
CI/CD	GitHub Actions, GitLab CI, or Azure DevOps	Automated build, test, migration, container image, deployment.

3. Frontend Comparison and UI Direction

Frontend	Strengths	Weaknesses	Decision
React + TypeScript	Best ecosystem, flexible, excellent for dashboards, easy to hire developers.	Needs good folder structure and standards.	Recommended.
Angular	Full enterprise framework, strict patterns.	More complex and heavier for smaller teams.	Good alternative, not necessary.
Vue	Easy and fast for UI.	Smaller enterprise ecosystem than React/Angular.	Good, but React is safer for hiring.
Next.js	Good for SEO and public patient portals.	More complexity for internal dashboards.	Use later for public site if needed.
Blazor	C# frontend, good .NET alignment.	Smaller frontend ecosystem and fewer templates.	Not recommended for first version.

Recommended frontend stack: React + TypeScript + Vite + Material UI/MUI X. Use React Router for navigation, TanStack Query for server state, React Hook Form for forms, Zod or Yup for validation, SignalR client for real-time updates, and Recharts/ECharts for dashboards.

3.1 Template Selection Rules

- Choose a template built with React and TypeScript, not old jQuery or plain Bootstrap admin templates.
- Prefer MUI, Ant Design, or Tailwind/shadcn templates with clean licensing for commercial use.
- Check that the template has table-heavy pages, form-heavy pages, dashboard cards, side navigation, top navigation, dark/light support, and responsive design.
- Do not let the template decide your architecture. Use the template only for UI components and layout inspiration.
- Avoid templates with too many dependencies, poor accessibility, or locked/unmaintained packages.

3.2 Suggested Visual Design: Calm Clinical OS

The design should feel calm, clean, and clinical. Hospital staff will use it all day, so it must be soothing to the eye, not overly colorful or noisy.

Design Area	Recommendation
Color palette	Warm white background, soft gray surfaces, deep teal/blue primary, green success, amber warning, red critical only when necessary.
Typography	Readable 14-16px body text, strong headings, high contrast for clinical screens.
Navigation	Left sidebar by module, top bar for search, notifications, quick patient

	lookup, and current department.
Dashboard style	Cards for KPIs, status chips for bed/queue/lab/pharmacy, timeline view for patient history.
Clinical screens	Patient banner at top, allergies/critical flags visible, tabs for history, notes, prescription, lab, billing.
Bed board	Floor-wise cards or grid with status colors: available, occupied, reserved, cleaning, maintenance, isolation.
Accessibility	Large clickable targets, keyboard navigation for heavy data entry, visible focus states.

4. Backend Comparison and Recommendation

Backend	Strengths	Weaknesses	Decision
ASP.NET Core Web API	Enterprise-ready, fast, secure, excellent SQL Server support, strong identity and reporting ecosystem.	AI ecosystem is weaker than Python.	Recommended main backend.
Java Spring Boot	Very mature enterprise backend, strong for large systems.	More verbose; slower for small teams.	Good alternative if team is Java-heavy.
NestJS	Structured TypeScript backend, good for real-time and modern APIs.	Microsoft SQL/reporting ecosystem weaker than .NET.	Good, but not your best fit.
Python FastAPI	Excellent for AI services, fast API development.	Not ideal as the only backend for large hospital business workflows unless team is Python-enterprise experienced.	Use for AI service, not core HMS.

Final backend decision: ASP.NET Core Web API for the hospital system, Python FastAPI for the AI service. This gives enterprise workflow strength and AI flexibility without mixing everything into one codebase.

4.1 Backend Module Structure

```

/src
  /Hospital.Api
  /Hospital.Application
  /Auth
  /Patients
  /Appointments
  /Queue
  /OPD
  /Emergency
  /Admissions
  /BedManagement
  /Nursing
  /Lab
  /Radiology
  /Pharmacy
  /Billing
  /Inventory
  /OT
  /Discharge
  /Reports
  /Notifications
  /Audit
  /Hospital.Domain
  /Hospital.Infrastructure
  /Hospital.Reporting
  /Hospital.Worker
  /Hospital.AI.Contracts

```

5. Database Strategy: SQL Server Now, Oracle-Ready Later

Recommendation: use Microsoft SQL Server for the first version and first production deployments. Keep the architecture clean enough to support Oracle later if a large enterprise client requires it.

Database	Best For	Strengths	Weaknesses	Decision
SQL Server	ASP.NET enterprise HMS, first production version.	Best .NET integration, EF Core provider, SSRS/Power BI alignment, strong transactions and reporting.	License cost; less natural if client is already Oracle-heavy.	Use now.
Oracle	Large enterprise or government-style hospitals with Oracle infrastructure.	Very powerful, mature, strong enterprise capabilities, vector/search features in newer versions.	Higher cost, more DBA expertise, slower development, more complex migrations.	Support later if required.
PostgreSQL	Modern open-source HMS if you are not committed to Microsoft.	Strong, cost-effective, very capable.	Not your chosen ecosystem; weaker Microsoft reporting alignment.	Good but not primary for you.
MongoDB	Flexible documents and unstructured data.	Good for document-style data.	Bad primary database for billing, admission, pharmacy stock, and audit transactions.	Do not use as main DB.

5.1 Why SQL Server Is the Better First Choice

- You are using ASP.NET Core, so SQL Server gives the smoothest data and reporting ecosystem.
- Development speed will be better for the first version because EF Core SQL Server support is very mature.
- SSRS and Power BI Report Server fit naturally into a Microsoft-backed reporting strategy.
- Hospital billing, pharmacy, admission, and audit logs are relational and transaction-heavy; SQL Server handles this well.
- Most performance problems will come from bad indexes, bad queries, no pagination, poor report design, and storing images wrongly - not from SQL Server itself.

5.2 How To Keep Oracle Migration Possible

- Do not put core business rules inside SQL Server stored procedures.
- Use EF Core for normal workflows, but isolate database provider logic in the Infrastructure layer.
- Use Dapper/stored procedures only for reporting and heavy queries, and keep those scripts versioned separately.
- Avoid SQL Server-only data types and functions unless necessary.
- Use public codes for patient/admission/invoice numbers instead of exposing database identity values.
- Keep schema naming simple and consistent so it maps better to Oracle later.

5.3 Data Access Rule

Use Case	Recommended Access
Normal create/update workflows	EF Core
Dashboards and fast read-only lists	Dapper
Heavy financial reports	Stored procedures or optimized SQL
Stock valuation and doctor commission	Stored procedures or Dapper
Large exports	Background job + optimized SQL
Medical image binary files	PACS/object storage, not the main relational DB

6. Medical Imaging and File Storage Strategy

The biggest database mistake in hospital software is storing all MRI/CT/X-ray images directly in the main business database. A hospital imaging system should use DICOM/PACS and object storage. The HMS database should store the metadata and links.

6.1 Imaging Storage Architecture

MRI/CT/X-ray/Ultrasound Device

-> DICOM/PACS Server

-> DICOMweb/Image Viewer

-> HMS Radiology Module

-> SQL Server stores patient link, order, report, study UID, status, and access permissions

-> Object storage/PACS stores actual image objects

Data Type	Where It Should Live
Patient demographics, orders, billing, reports	SQL Server
DICOM image files and large scan objects	PACS + object storage
PDF reports, consent forms, insurance documents	Object storage such as MinIO/S3/Azure Blob
Image viewing	DICOM viewer connected to PACS/DICOMweb

Radiology report text and approval status	SQL Server
---	------------

For an MVP, you can start by uploading radiology report PDFs and image references. For a serious hospital deployment, integrate with a PACS/DICOM server and use a proper viewer. Open-source Orthanc can be evaluated for early development/testing, but production hospital imaging may require a commercial PACS/VNA depending on the hospital.

7. Reporting System Strategy

Reports will strongly affect the perceived value of the software. A hospital management system is judged not only by data entry screens but by the quality, speed, accuracy, printability, and usefulness of reports.

7.1 Four-Layer Reporting Architecture

Reporting Layer	Tools	Examples	Notes
Transactional documents	QuestPDF or HTML-to-PDF	Prescription, invoice, money receipt, discharge summary, lab report, consent form.	Generated from exact transaction data.
Operational reports	SSRS / Power BI Report Server / FastReport	Daily collection, bed occupancy, lab workload, pharmacy sale, due report.	Printable, filterable, exportable.
Analytics dashboards	React dashboards + Power BI later	Revenue trends, patient flow, occupancy, stock trend, department performance.	Use cached/summary data for speed.
AI narrative reports	Python FastAPI AI service + approved SQL data	Monthly executive summary, anomaly explanation, AI-written management narrative.	AI summarizes data; AI should not invent numbers.

7.2 Recommended Reporting Stack

- QuestPDF for code-first, pixel-consistent documents such as invoice, prescription, receipt, discharge summary, and lab report.
- SSRS or Power BI Report Server for enterprise paginated reports where management needs filters, export, scheduled delivery, and printing.
- FastReport or Stimulsoft if you want a more visual report designer embedded into your product.
- Dapper and stored procedures for heavy reporting queries.
- A reporting database or warehouse later so heavy reports do not slow down live patient operations.
- AI report writer at the end: SQL generates numbers, AI writes explanations, highlights, summaries, and recommendations.

7.3 AI-Generated Reports Rule

AI should not be the source of truth for report numbers. The database/report engine must calculate all figures. AI can then write a human-friendly explanation: why revenue increased, why bed occupancy dropped, which departments are overloaded, which stocks may run out, or what unusual billing patterns were detected.

Report Type	Data Source	AI Role
Daily cash closing	SQL Server report query	No AI needed except anomaly comments.
Monthly management report	Reporting database/SQL summaries	AI writes executive summary and highlights.
Doctor performance report	SQL summaries	AI explains trends carefully.
Pharmacy stock forecast	Stock movement history	AI/ML predicts demand and suggests reorder review.
Clinical discharge summary	Patient record and doctor notes	AI drafts, doctor approves.
Lab result explanation	Approved lab result	AI explains in patient-friendly language with safety disclaimers.

8. AI Technology Stack and AI Phase

AI should be a separate service, added after the core hospital management system is stable. This keeps clinical safety, auditing, model changes, and experimentation separated from the main hospital workflow.

8.1 AI Architecture

React UI

- > ASP.NET Core API checks permission
- > ASP.NET Core collects approved patient/report data
- > ASP.NET Core sends minimal required data to Python FastAPI AI Service
- > AI Service calls OpenAI/Azure OpenAI/local model
- > AI Service returns draft/summary/alert
- > Doctor/admin reviews
- > Approved result is saved with audit log and model version

AI Component	Recommended Tool
AI API service	Python FastAPI
LLM provider first	Azure OpenAI or OpenAI API depending on compliance/client preference
Local model later	Use only after GPU/infrastructure, validation, and quality testing are ready
Embeddings/vector search	Qdrant, Azure AI Search, or database-native vector features later
Medical imaging AI research	MONAI/PyTorch only after imaging workflow and validation plan exist
Prompt/version logging	SQL Server AI audit tables + object storage for large artifacts
AI safety	Human approval workflow, disclaimers, model/version record, feedback capture

8.2 AI Features by Phase

AI Feature	Risk Level	When To Build
AI discharge summary draft	Low to medium	First AI phase after IPD/discharge works.
AI doctor note structuring	Low to medium	First AI phase after OPD notes exist.
AI lab result explanation for patients	Medium	After lab reports and patient portal are stable.
AI duplicate patient detection	Low	Can be added early using deterministic + ML matching.
AI billing anomaly detection	Medium	After billing data volume is enough.
AI stock prediction	Medium	After several months of pharmacy/inventory data.
AI diagnosis suggestions	High	Later only with medical governance and clinician approval.
AI radiology image diagnosis	Very high	Specialist project, validated model, regulatory review, radiologist approval.

8.3 AI Safety Rules

- AI output is always a draft unless a responsible human approves it.
- AI never silently edits medical records.
- Every AI output stores model name, prompt version, input data reference, output, user feedback, approval status, and audit log.
- Do not send unnecessary identifiers to external AI providers.
- Use clinical disclaimers for patient-facing explanations.
- Do not start high-risk AI diagnosis/radiology features before legal, clinical, and validation review.

9. Full Hospital Workflow

The software should follow the real hospital journey. Every module should be validated with hospital staff before coding heavily.

9.1 Patient Registration and Appointment

- Patient arrives, calls, or books online.
- Reception searches existing patient or creates new profile.
- Patient ID is assigned.
- Department/doctor/service is selected.
- Appointment or walk-in visit is created.
- Payment is collected if required.

- Token/queue number is generated.
- Patient is guided to doctor, floor, or department.

9.2 OPD Workflow

- Patient checks in and waits in queue.
- Nurse/assistant records vitals if needed.
- Doctor opens patient record and past history.
- Doctor records complaint, examination, diagnosis, advice.
- Doctor creates prescription.
- Doctor orders lab/radiology if needed.
- Patient completes test/payment/pharmacy workflow.
- Follow-up appointment is created if needed.

9.3 Emergency Workflow

- Emergency patient arrives.
- Fast registration with minimal details.
- Triage nurse records urgency and vitals.
- Emergency doctor/nurse starts immediate care.
- Lab/radiology/medicine/procedure orders are made.
- Outcome: discharge, admission, ICU, OT, referral, death case, or LAMA/DAMA.

9.4 Admission and IPD Workflow

- Doctor advises admission.
- Admission desk checks bed availability and room type.
- Deposit is collected if required.
- Bed/room/floor is assigned.
- Floor reception and nurse station are notified.
- Nurse confirms patient arrival and records initial vitals.
- Doctor rounds, nursing care, medication, lab/radiology, and billing continue until discharge.

9.5 Building, Floor, Room, and Bed Workflow

- Hospital has buildings, floors, floor receptions, nurse stations, rooms, wards, beds.
- Every bed has status: available, occupied, reserved, cleaning required, under cleaning, maintenance, isolation, blocked.
- Admission assigns bed; transfer changes bed; discharge sends bed to cleaning; housekeeping releases bed after cleaning.
- Floor reception coordinates arrival, visitors, transfer, cleaning, and discharge movement.

9.6 Nursing Workflow

- Nurse sees assigned floor/ward/room patients.
- Nurse records vitals and nursing notes.
- Nurse follows doctor orders.
- Nurse gives medicine according to schedule and records given/missed/refused/held.
- Nurse reports critical changes.
- Nurse performs shift handover with pending tasks.

9.7 Lab Workflow

- Doctor orders lab test.
- Payment is checked if required.
- Sample collection team collects and labels sample.
- Lab processes test and enters result.
- Technician/pathologist verifies and approves.

- Critical values notify doctor/nurse.
- Report becomes available to doctor/patient.

9.8 Radiology Workflow

- Doctor orders imaging.
- Radiology schedules scan.
- Patient preparation happens if required.
- Scan is performed.
- Images are stored in PACS/object storage.
- Radiologist writes/approves report.
- Doctor/patient can view report; billing is updated.

9.9 Pharmacy Workflow

- Pharmacist receives OPD prescription or IPD medicine request.
- Medicine availability and batch/expiry are checked.
- Medicine is sold/issued.
- Stock is deducted.
- IPD medicine is added to running bill.
- Unused medicine return adjusts stock and bill if approved.

9.10 Billing Workflow

- OPD bills are created per service.
- IPD billing runs continuously from bed, doctor, lab, radiology, medicine, OT, procedures, oxygen, diet, and other services.
- Discounts require approval.
- Refunds require reason and approval.
- Daily cash closing is completed by cashier and accounts.

9.11 Operation Theater Workflow

- Doctor advises surgery.
- Consent, pre-op tests, anesthesia assessment, and OT booking are completed.
- Nurse prepares patient using checklist.
- Surgery is performed and OT note is recorded.
- Used medicines/items update billing/inventory.
- Patient moves to recovery/ICU/ward.

9.12 Discharge Workflow

- Doctor approves discharge.
- Nurse starts discharge preparation.
- Pharmacy return/reconciliation is done.
- Pending lab/radiology/billing items are checked.
- Final bill is paid/cleared.
- Doctor approves discharge summary.
- Patient leaves.
- Bed goes to cleaning and later becomes available.

9.13 Admin and Reporting Workflow

- Admin monitors OPD, emergency, IPD, bed occupancy, ICU, revenue, due amount, lab workload, pharmacy stock, and staff performance.
- Management gets daily, weekly, monthly, department-wise, doctor-wise, and financial reports.
- AI summaries are generated only from approved report data later.

10. Full Software Architecture

10.1 High-Level Architecture

Users

- > React Web App
- > ASP.NET Core API
 - > SQL Server operational database
 - > Redis cache
 - > SignalR real-time hub
 - > Hangfire workers
 - > Object storage
 - > Reporting service
 - > PACS/DICOM server
 - > Notification providers
 - > Python FastAPI AI service later

10.2 Core Modules

- Auth and identity
- User roles and permissions
- Patient management
- Appointment and queue
- OPD
- Emergency
- Admission
- Building/floor/room/bed
- Nurse station
- Floor reception
- IPD care
- Lab
- Radiology
- Pharmacy
- Billing and accounts
- Inventory and purchase
- OT/surgery
- Discharge
- Housekeeping
- Maintenance
- Visitor management
- Ambulance
- Reports
- Notifications
- Audit logs
- Settings
- AI service later

10.3 Data and Storage Architecture

Storage	Purpose
SQL Server operational DB	Main HMS transactions: patients, visits, admissions, billing, stock, audit, reports metadata.
Object storage	PDFs, documents, consent files, patient uploads, scanned reports, signatures.
PACS/DICOM server	Medical imaging studies and DICOM image retrieval/viewing.
Redis	Cache, queue counters, short-lived tokens, real-time status acceleration.
Reporting database/warehouse later	Read-optimized summaries and analytics without slowing operational DB.
AI logs and vector storage later	AI inputs/outputs, prompt versions, embeddings, knowledge search.

10.4 API and Integration Architecture

- Use REST APIs first with Swagger/OpenAPI documentation.
- Use SignalR for real-time queue, bed board, lab/pharmacy/billing notifications.
- Use background workers for SMS/email, report generation, data exports, and AI tasks.
- Use FHIR mapping concepts for future interoperability: Patient, Practitioner, Encounter, Observation, MedicationRequest, DiagnosticReport, Appointment.
- Use DICOM/DICOMweb concepts for imaging integration when radiology matures.
- Keep external integrations behind adapter interfaces so vendors can change without rewriting business modules.

11. Security, Compliance, and Audit Design

- Use ASP.NET Core Identity or a custom identity module built on secure password hashing, JWT access tokens, refresh tokens, and MFA for high-risk users.
- Use role-based access control plus permission-based access control. Roles alone are not enough for hospital software.
- Every patient record view/edit, prescription edit, lab approval, discount, refund, user permission change, discharge approval, and AI generation must create an audit log.
- Use HTTPS/TLS everywhere, encryption at rest where possible, secure backups, and no real patient data in development environments.
- Use separate development, staging, and production environments.
- Use secrets management for database strings, AI keys, SMS keys, and storage keys.
- Build data retention, backup, and disaster recovery rules from the start.
- AI outputs must be labeled, auditable, and human-approved for clinical use.

Security Area	Recommended Control
Authentication	JWT + refresh tokens + MFA for admins/doctors/high-risk roles.
Authorization	Roles + fine-grained permissions.
Patient privacy	Minimum necessary access, audit logs, patient context banner.
Billing safety	Approval workflow for discounts/refunds/write-offs.
Clinical safety	Versioned prescriptions, reports, discharge summaries, doctor approval.
Infrastructure	TLS, firewall, network segmentation, backups, monitoring.
AI safety	De-identification where possible, human review, model/prompt version logging.

12. DevOps, Hosting, Docker, and Kubernetes Plan

12.1 Development and MVP Hosting

- Use Docker Compose for local development: API, frontend, SQL Server dev instance, Redis, object storage, worker, AI mock service later.
- Use Windows Server + IIS or Linux containers behind Nginx/Ingress depending on team skill and hosting choice.
- Use SQL Server on a dedicated database server or managed database, not casually inside Kubernetes for production unless the team has strong database operations expertise.
- Use object storage with persistent volumes or managed cloud storage.
- Use CI/CD to run tests, build containers, apply migrations carefully, and deploy to staging first.

12.2 Kubernetes Plan

Kubernetes is useful when the application has multiple services, workers, high availability, and scaling needs. It is not mandatory on day one. Start with Docker and clean deployment scripts. Move to Kubernetes when staging is stable or when the first serious hospital deployment requires high availability.

Stage	Hosting Recommendation
Development	Docker Compose on developer machines.
Internal staging	Docker Compose or small Kubernetes cluster.
First production	Dedicated VM/Windows Server/IIS or containerized deployment with managed DB.

Enterprise production	Kubernetes for API, frontend, worker, reporting service, AI service, notification service; dedicated/managed DB and storage.
-----------------------	--

13. Step-by-Step Build Phases and Estimated Timeline

The timeline below assumes a small team using Codex heavily for scaffolding, CRUD screens, DTOs, tests, refactoring, and documentation. A solo developer may take longer. A strong team may compress the timeline. The safest plan is to build core HMS first and AI last.

Phase	Estimated Duration	Main Deliverables
0. Hospital validation	2-3 weeks	Confirm workflows with reception, doctors, nurses, lab, pharmacy, billing, admin, floor reception, housekeeping. Finalize modules and forms.
1. Project foundation	2 weeks	Solution setup, React app, ASP.NET projects, CI/CD, Docker Compose, coding standards, database baseline, auth skeleton.
2. Auth, users, roles, settings	2-3 weeks	Login, roles, permissions, departments, doctors, service pricing, audit log foundation.
3. Patient, appointment, queue	3-4 weeks	Patient registration, duplicate check, appointment booking, token queue, reception dashboard.
4. OPD and prescription	4-5 weeks	Doctor dashboard, consultation, vitals, diagnosis, prescriptions, follow-up, print prescription.
5. Billing core	4-5 weeks	OPD billing, payments, receipts, discounts approval, refunds, daily cash closing.
6. Lab and radiology basic	4-6 weeks	Lab orders, sample collection, result entry, approval, report print; radiology orders and report upload.
7. Pharmacy and inventory	5-6 weeks	Medicine master, batch/expiry, sales, IPD issue, returns, purchase request, stock reports.
8. Building/floor/room/bed/IPD/nursing	6-8 weeks	Hospital structure, bed board, admission, bed assignment/transfer, nurse station, medication administration, doctor rounds.
9. Emergency, OT, discharge	6-8 weeks	Emergency registration/triage, OT scheduling/checklists, discharge process, final bill, discharge summary, death/referral/LAMA.
10. Reporting and dashboards	4-6 weeks	SSRS/FastReport/QuestPDF reports, admin dashboards, export, scheduled reports, KPI summaries.
11. Hardening and pilot	4-6 weeks	Performance testing, security review, backup/restore, audit review, bug fixing, hospital pilot training.
12. AI phase	6-10 weeks first AI release	AI note assistant, discharge draft, lab explanation, report narrative, anomaly detection foundation, AI audit logs.

13.1 Suggested MVP Boundary

- Do not build everything before testing with real users.
- First MVP should include patient registration, appointments, queue, OPD, prescription, billing, lab basics, pharmacy basics, reports, audit logs.
- Second milestone should include admission, floor/room/bed, IPD, nursing, and discharge.
- Third milestone should include emergency, OT, advanced reporting, inventory, patient portal.
- AI should start only after stable data capture exists.

14. Rules for Codex-Assisted Development

Since you plan to use Codex, treat Codex as a coding agent, not the architect of the whole system. Give it small phase-based tasks with strict architecture rules, acceptance criteria, test requirements, and file boundaries.

- Create one phase prompt at a time.
- Require Codex to preserve Clean Architecture boundaries.
- Require migrations, DTOs, validators, unit tests, API tests, and seed data for each module.
- Never let Codex skip audit logging for sensitive workflows.
- Never let Codex create direct SQL everywhere without approval.
- Require clear error handling and permission checks in every endpoint.

- Require frontend screens to match the design system and role permissions.
- Run tests and review generated code before moving to the next phase.

14.1 Codex Prompt Files After Confirmation

Per your instruction, phase-by-phase Codex coding-agent prompt files are not included in this PDF yet. After you confirm this blueprint, the next deliverable should be a set of prompt files such as:

- phase_00_requirements_and_domain_model_prompt.md
- phase_01_solution_foundation_prompt.md
- phase_02_auth_roles_permissions_prompt.md
- phase_03_patient_appointment_queue_prompt.md
- phase_04_opd_prescription_prompt.md
- phase_05_billing_core_prompt.md
- phase_06_lab_radiology_prompt.md
- phase_07_pharmacy_inventory_prompt.md
- phase_08_ipd_bed_nursing_prompt.md
- phase_09_emergency_ot_discharge_prompt.md
- phase_10_reporting_dashboards_prompt.md
- phase_11_security_performance_pilot_prompt.md
- phase_12_ai_service_prompt.md

15. Final Decisions To Confirm Before Coding

Decision	Recommended Answer
Frontend	React + TypeScript + Vite + MUI/MUI X, using online template only as UI base.
Backend	ASP.NET Core Web API on current .NET LTS.
Architecture	Clean Architecture + Modular Monolith first.
Database now	SQL Server.
Oracle future	Keep Oracle-ready architecture, support Oracle later only if client requires it.
Image storage	PACS/DICOM + object storage; not relational DB blobs.
Data access	EF Core for workflows, Dapper for read reports, stored procedures for heavy reports.
Reporting	QuestPDF + SSRS/Power BI Report Server; FastReport/Stimulsoft optional.
AI	Python FastAPI service after core HMS, with human approval and audit.
Docker/Kubernetes	Docker from day one; Kubernetes when staging/enterprise deployment needs it.

If these decisions are approved, the next step is to generate the Codex phase prompt files and then start with Phase 0 and Phase 1.

16. References

- Microsoft ASP.NET Core documentation: <https://learn.microsoft.com/en-us/aspnet/core/>
- Microsoft .NET support policy: <https://dotnet.microsoft.com/en-us/platform/support/policy>
- Microsoft EF Core SQL Server provider: <https://learn.microsoft.com/en-us/ef/core/providers/sql-server/>
- Microsoft SQL Server 2025 documentation: <https://learn.microsoft.com/en-us/sql/sql-server/what-s-new-in-sql-server-2025>
- Oracle ODP.NET EF Core provider: <https://docs.oracle.com/en/database/oracle/oracle-database/26/odpnt/ODPEFCore.html>
- Oracle AI Database documentation: <https://docs.oracle.com/en/database/oracle/oracle-database/>
- React documentation: <https://react.dev/learn>
- MUI documentation: <https://mui.com/>
- DICOM Standard: <https://www.dicomstandard.org/>
- HL7 FHIR overview: <https://fhir.hl7.org/fhir/overview.html>

- SQL Server Reporting Services reports: <https://learn.microsoft.com/en-us/sql/reporting-services/reports/reporting-services-reports-ssrs>
- Power BI Report Server paginated reports: <https://learn.microsoft.com/en-us/power-bi/report-server/quickstart-create-paginated-report>
- QuestPDF documentation: <https://www.questpdf.com/>
- Docker documentation: <https://docs.docker.com/>
- Kubernetes documentation: <https://kubernetes.io/docs/home/>
- FastAPI documentation: <https://fastapi.tiangolo.com/>
- OpenAI API documentation: <https://developers.openai.com/api/docs>
- Azure OpenAI REST API reference: <https://learn.microsoft.com/en-us/azure/foundry/openai/reference>
- MONAI medical imaging AI framework: <https://project-monai.github.io/>
- WHO AI ethics and governance guidance for health: <https://www.who.int/news/item/18-01-2024-who-releases-ai-ethics-and-governance-guidance-for-large-multi-modal-models>
- HHS HIPAA Security Rule: <https://www.hhs.gov/hipaa/for-professionals/security/index.html>